Deploying a Dockerized Node App for Customer Portals

Skills Network

Estimated time needed: 30 minutes

In this lab, you will set up and deploy a Node.js application coupled with MongoDB on Docker. Your objective is to replicate the customer portal repository utilized in previous exercises and commence the deployment of the customer portal application using Docker.

Learning Objective:

After completing this lab, you will be able to:

- Clone a Customer portal Node.js application from a GitHub repository
- Complete configuration of Dockerfile and docker-compose.yml
- Deploy the Node.js application on Docker

About Skills Network Cloud IDE

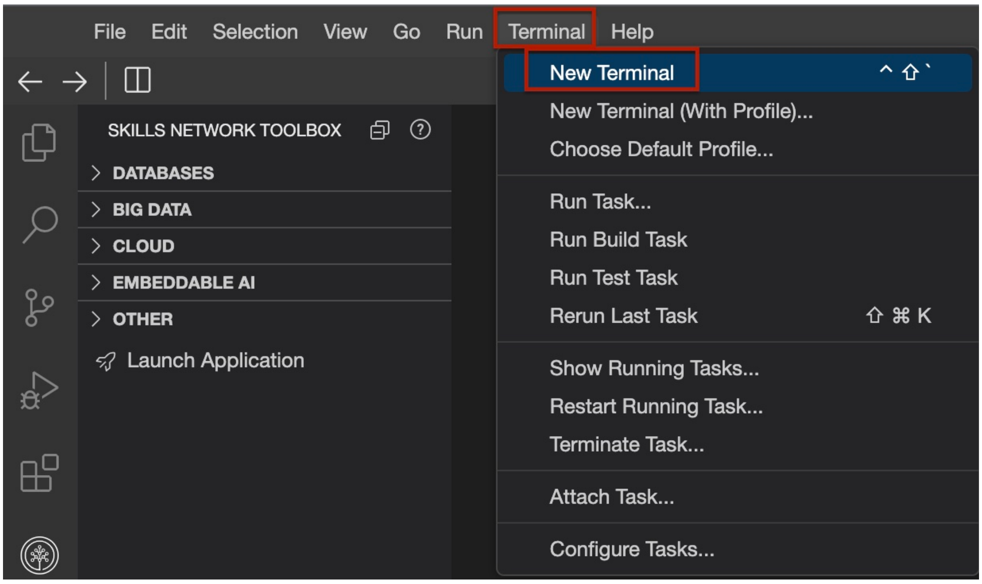
Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. Theia is an open source IDE (Integrated Development Environment).

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

Task 1: Clone a Customer portal Node.js application from a GitHub repository

1. Open a terminal window by using the menu in the editor: Terminal > New Terminal.



2. Change to your project folder, if you are not in the project folder already.

```
cd /home/project
```

3. In the terminal, clone the git repository which has the starter code.

```
git clone https://github.com/ibm-developer-skills-network/abltc-backend-nodejs-customerportal.git
```

4. Change to the new directory that you just cloned.

```
cd abltc-backend-nodejs-customerportal
```

5. In the terminal, run the following command to install the node packages required.

```
npm install
```

Task 2: Configuration of Dockerfile and docker-compose.yml

1. Create an Dockerfile file using the command below.

```
touch Dockerfile
```

2. Paste the below code into Dockerfile.

```
FROM node:18.12.1-bullseye-slim
# Set the working directory in the container
WORKDIR /app
# Copy package.json and package-lock.json to the working directory
COPY package*.json ./
# Install app dependencies
RUN npm install
# Copy necessary files
COPY . .
# Expose port 3000 (adjust the port based on your application)
EXPOSE 3000
# CMD to run the application
CMD [ "node", "customer_app.js" ]
```

3. Create an docker-compose.yml file using below command.

```
touch docker-compose.yml
```

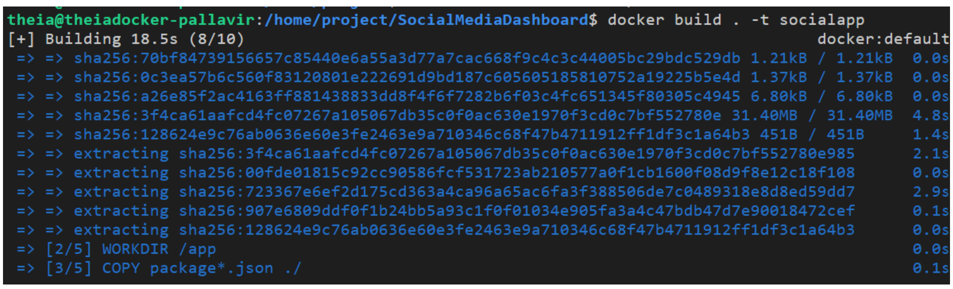
4. Paste the below code into docker-compose.yml

```
version: '3'
services:
  mongodb:
    image: mongo
    ports:
      - "27017:27017"
    networks:
      - app-network

  app:
    image: customerapp
    ports:
      - "3000:3000"
    depends_on:
      - mongodb
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

Task 3: Deploy the Node.js application on Docker

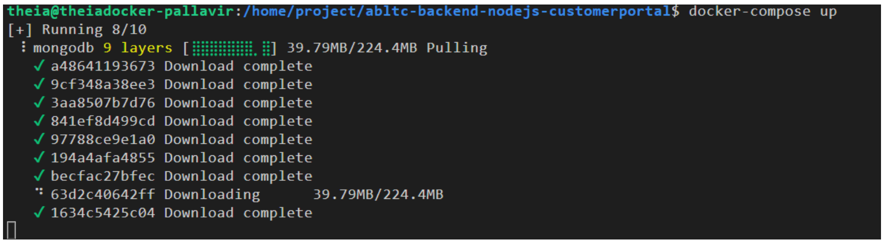
1. Run the following command to build the Docker app

```
docker build . -t customerapp
```



2. The docker-compose.yml has been created to run two containers, one for Mongo and the other for the Node app. Run the following command to run the server:

```
docker-compose up
```

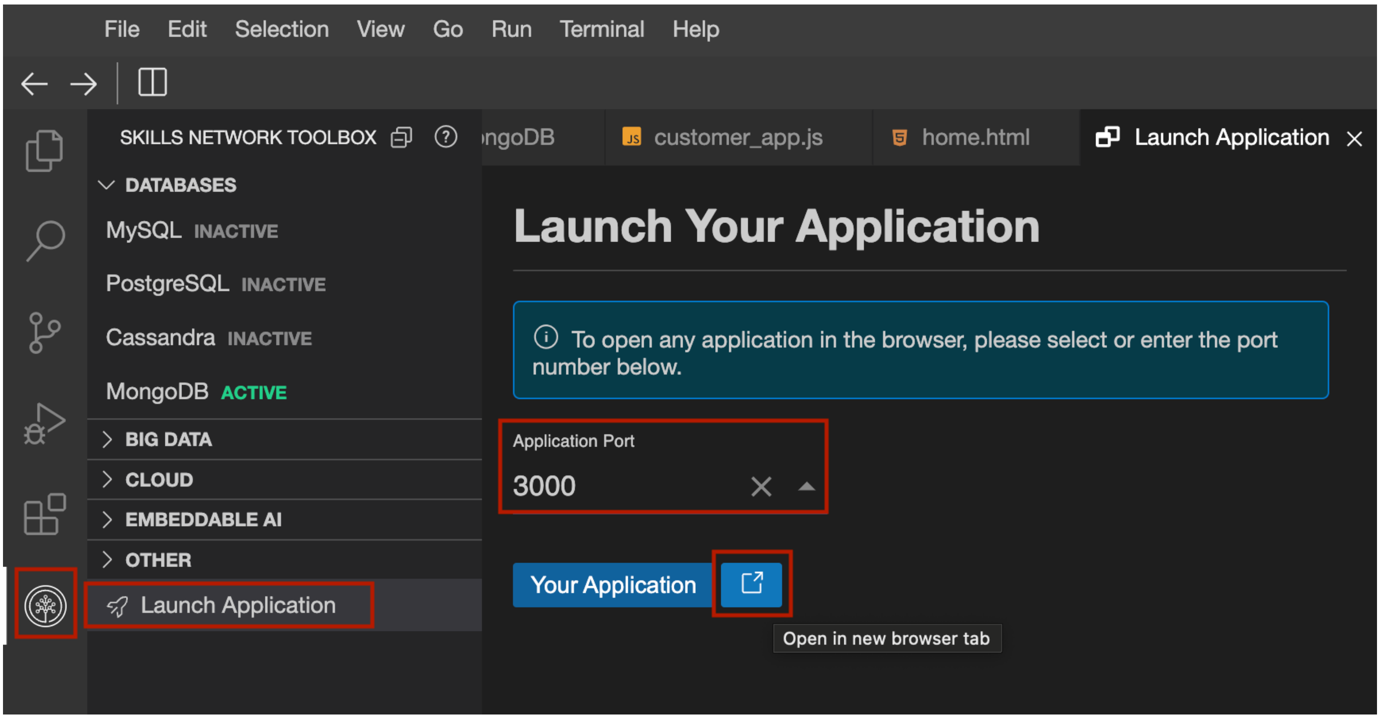


3. Click on the button below to launch the application.

Customer Portal

▾ Click here if the button above doesn't work

1. Click on the `Skills Network` icon in the left tool bar.
2. Choose `Launch Application`.
3. Enter the port number `3000` on which the application is running.
4. Click on the icon, as shown in the image, to open the customer portal in a new tab.



Authors

Pallavi Rai